

Session 3 Speaker Script — COMPLETE WITH FULL TALKING POINTS

Prompt Engineering Fundamentals

Presenter: Dinesh Wadhvani | **Session:** 3 of 12 | **Duration:** ~60 minutes

Purpose: Master the art of writing prompts that reliably produce better outputs

Slide	Title	Time
1	Title Slide	~1 min
2	Today's Journey	~2 min
3	What is Prompt Engineering?	~5 min
4	Prompt Structure Fundamentals	~8 min
5	Clarity & Specificity	~8 min
6	Few-Shot Learning (Examples)	~8 min
7	Common Pitfalls	~8 min
8	Real-World Examples	~5 min
9	Prompt Templates & Patterns	~5 min
10	Lab Overview	~3 min
11	Key Takeaways	~5 min
12	Session 4 Preview	~2 min
—	Lab Exercise	~12 min

01. TITLE SLIDE

~1 minute

WHAT'S ON SCREEN

Session 3 label (blue) • Title: 'Prompt Engineering Fundamentals' • Session info: 'Session 3 of 12 | 60 minutes' • Dark blue background

SPEAKING SCRIPT

Welcome back to Session 3! Last session, we looked under the hood at how LLMs actually work. We learned about tokens, transformers, attention mechanisms, context windows, and parameters. We developed deep understanding of the mechanics.

Now comes the practical application: how to write prompts that reliably produce better outputs. This is where the rubber meets the road. All the knowledge from Session 2—about tokenization, attention, temperature—we're going to apply strategically.

Here's the truth: the difference between a mediocre prompt and an excellent prompt can be the difference between a useless answer and a brilliant one. And the techniques aren't magical. They're learnable, repeatable, systematic.

By the end of today, you'll know how to structure prompts, use examples strategically, avoid common pitfalls, and—most importantly—think about prompt engineering as a discipline with real principles and patterns. You'll take your Session 1 resume and transform it into something production-ready.

Let's begin.

DELIVERY TIPS

Set high energy—this is practical and immediately useful. • Emphasize that prompt engineering is a learnable skill, not magic. • Quick question: 'Anyone tried tweaking a prompt and got wildly different results?' Sets context.

02. TODAY'S JOURNEY

~2 minutes

WHAT'S ON SCREEN

12-item agenda with time badges • Items: Recap, What is PE, Structure, Clarity, Examples, Pitfalls, Real Examples, Templates, Lab, Takeaways, Session 4 Preview

SPEAKING SCRIPT

Quick roadmap of our hour. We start with a five-minute recap—connecting Session 2 concepts to prompt engineering. Why does tokenization matter for prompts? Why does attention matter? We're bridging knowledge to practice.

Then eight minutes on what prompt engineering actually is. It's not magic incantations. It's systematic technique grounded in how LLMs work.

Eight minutes on prompt structure. There are patterns that work reliably. We'll see them.

Eight minutes on clarity and specificity. Vague prompts get vague answers. Specific prompts get specific answers.

Eight minutes on few-shot learning—why showing examples is often more powerful than explaining.

Eight minutes on common pitfalls. What kills prompts? What makes them fail?

Five minutes on real-world examples. Actual prompts, actual results. We'll see what works in practice.

Five minutes on templates and patterns you can reuse.

Then the lab—about 12 minutes. You'll take your Session 1 resume and craft it into a truly professional prompt. Iterate. Test. Refine. Document your decisions.

Then key takeaways and Session 4 preview. Notice the progression: concept, structure, application, then hands-on. If time gets tight, we compress the examples section first. We protect the lab.

DELIVERY TIPS

Move briskly through roadmap. • Emphasize that students will have concrete, reusable templates by end of session. • This is immediately applicable material.

03. WHAT IS PROMPT ENGINEERING?

~5 minutes

WHAT'S ON SCREEN

Definition box • 'The discipline of crafting prompts to reliably produce desired outputs' • Three pillars: Structure, Clarity, Context • Real-world impact stats

SPEAKING SCRIPT

Prompt engineering is the discipline of crafting prompts to reliably produce desired outputs from language models. It's not trying random things and hoping something works. It's systematic.

Think about it this way: Last session, we learned that LLMs work by predicting the next token. What token comes next depends entirely on patterns the model learned from training and on what the attention mechanism decides matters in YOUR input.

Prompt engineering is about: how do I frame my input so the attention mechanism focuses on what matters? How do I structure my request so the model's learned patterns activate in the right way? How do I remove ambiguity so the model's predictions converge toward what I want?

Here's the striking part: companies are spending massive money on prompt engineering teams. Why? Because a well-engineered prompt for a customer support chatbot, a sales email generator, a code generator—can dramatically reduce error rates and improve quality.

Real numbers: A mediocre prompt to an LLM might get 60% accuracy. A well-engineered prompt to the same LLM might get 95% accuracy. Same model. Different prompt. That's the power of prompt engineering.

There are three pillars to good prompt engineering: First, structure—how you organize your request. Second, clarity—removing ambiguity. Third, context—providing enough information for the model to reason properly.

Everything we talk about today is grounded in one simple fact: LLMs are pattern-matching systems. We're learning to activate the RIGHT patterns by asking in the right way.

DELIVERY TIPS

Emphasize this is a learnable discipline, not magic. • The 60% to 95% accuracy stat is powerful—use it. • Connect back to Session 2 concepts (attention, patterns). • Real-world examples of prompt engineering teams signal importance.

04. PROMPT STRUCTURE FUNDAMENTALS

~8 minutes

WHAT'S ON SCREEN

Four-part structure diagram • Role/Context → Task → Constraints → Output Format • Real example with before/after • Visual showing improvement

SPEAKING SCRIPT

There's a pattern to prompts that work well. It's not the only pattern, but it's incredibly reliable. I call it the four-part structure: Role, Task, Constraints, Output Format.

Let me show you why each matters, using real examples.

PART 1: ROLE / CONTEXT

Start by telling the model what role to play or what context to work within. Not always necessary, but incredibly powerful when used strategically.

Example—bad prompt: 'Explain photosynthesis.'

Example—better prompt: 'You are a high school biology teacher explaining photosynthesis to students who've never heard of it. Explain photosynthesis.'

Why the difference? Because when you specify a role, the model activates different patterns. 'Teacher explaining to beginners' activates patterns about simplicity, analogies, patience. 'Explain photosynthesis' to the general model—it might activate patterns from dense academic papers.

Real-world example: A company hiring customer service reps fine-tunes models by starting every prompt with 'You are a helpful, empathetic customer service representative.' This single framing dramatically improves tone and empathy in responses.

PART 2: TASK

What specifically do you want the model to do? Be explicit. Don't assume the model will infer.

Bad: 'Tell me about climate change.'

Better: 'Summarize the top three ways climate change is affecting ocean ecosystems in 200 words or less.'

The second version is specific. 'Top three ways' narrows it. 'Ocean ecosystems' narrows it. '200 words or less' narrows it. The model knows exactly what to produce.

PART 3: CONSTRAINTS

What limitations or requirements should guide the response? Constraints improve focus and quality.

Constraints might be: Length (200-300 words), tone (professional, casual, academic), format (bullet points, narrative, JSON), audience (5-year-old, executive, expert), restrictions (avoid jargon, avoid technical terms, focus only on X, don't mention Y).

Example: 'Explain machine learning to a 10-year-old. Use only everyday words. Keep it to 150 words. Don't mention math.'

Compare to: 'Explain machine learning.'

Massive difference in output quality and usefulness.

PART 4: OUTPUT FORMAT

Tell the model explicitly what format you want. This is powerful and often overlooked.

Without format: 'List the best productivity tools.'

With format: 'List the best productivity tools. Format as a numbered list. For each tool, include: name, primary use, price, best for (one sentence).'

When you specify format, the model knows exactly what to produce. It generates more consistently. It's easier for you to parse the output.

Real example: Companies building AI workflows that connect tools together (AI generating data → another tool processing it) require very specific output formats. A well-formatted prompt can mean the difference between data that downstream tools can parse and data that breaks the pipeline.

PUTTING IT TOGETHER: Before and After

Bad prompt: 'Write a resume for a student.'

Better prompt: 'You are a professional resume writer helping a recent graduate. Write a one-page resume for a final-year Computer Science student named Priya Sharma with skills in Python, SQL, and data visualization, and a summer internship at TechCorp building a customer analytics dashboard. Format it professionally with clear sections: Summary, Skills, Experience, Education, Projects. Use action verbs and quantifiable metrics where possible. Output as plain text.'

The second prompt is longer but dramatically more useful. The model knows exactly what to do. The output is much more likely to be what you actually want.

DELIVERY TIPS

Use the four-part structure consistently throughout the session. • Show before/after comparisons—the contrast is powerful. • Real-world example of AI pipeline tools shows why format matters. • This structure is immediately applicable—students will use it today in lab.

05. CLARITY & SPECIFICITY

~8 minutes

WHAT'S ON SCREEN

Clarity spectrum diagram • Vague ← → Specific • Real examples showing output differences • Metrics and measurable criteria highlighted

SPEAKING SCRIPT

Here's a principle that runs through all good prompt engineering: clarity and specificity dramatically improve output quality. It's almost impossible to be too specific.

THE VAGUENESS PROBLEM

When prompts are vague, the model's attention mechanism doesn't know what matters. Multiple patterns activate. Output becomes unfocused, mediocre, sometimes wrong.

Example: 'Write a nice essay about AI.'

Problems with this prompt: What does 'nice' mean? What audience? What length? What aspects of AI? What level of technical detail? The model has to guess. And different runs might produce different results because the patterns aren't constrained.

THE SPECIFICITY SOLUTION

Specific prompts constrain the model's attention. The right patterns activate. Output becomes focused and usable.

Same request, made specific: 'Write a 1,500-word essay about how machine learning is changing healthcare, aimed at healthcare executives with non-technical backgrounds. Focus on practical applications and ROI, not theory. Use clear language, avoid jargon, include 2-3 real-world examples. Structure: introduction, three main sections (diagnosis, treatment planning, patient monitoring), conclusion.'

Now the model knows exactly what to produce. The output is almost certainly better.

SPECIFICITY IN PRACTICE: Real Examples

Let me show you how specificity works across different use cases.

USE CASE 1: Writing Code

Vague: 'Write code to sort a list.'

Better: 'Write a Python function that sorts a list of dictionaries by the 'date' field in descending order (newest first). Include error handling for missing date fields. Include a docstring with example usage.'

The second version is specific about language, data structure, sorting direction, error handling, documentation. The output will be exactly what you need.

USE CASE 2: Customer Service Response

Vague: 'Write a response to a customer complaint.'

Better: 'Write a customer service response to someone complaining that their order arrived three days late. The customer is frustrated but not angry. Acknowledge the delay, apologize sincerely, offer a 15% discount on their next order, and ask if there's anything else we can do. Keep it to 100-150 words. Tone: professional but warm.'

Specific = consistent quality, appropriate tone, right length, right content.

USE CASE 3: Data Analysis

Vague: 'Analyze this data.'

Better: 'Analyze this sales data and identify trends. Specifically: 1) What's the revenue trend over the past 12 months? Is it growing or declining? 2) Which product categories are growing fastest? 3) Which geographic regions underperformed? For each finding, give me a one-sentence summary plus the supporting number. Format as bullet points.'

HOW TO MAKE A PROMPT SPECIFIC

Use concrete details instead of abstract concepts. Instead of 'nice', use 'professional, warm, and conversational'. Instead of 'interesting', use 'includes at least 3 surprising facts'. Instead of 'good quality', use 'free of grammatical errors, uses active voice, uses strong verbs'.

Define your criteria measurably. Not 'long enough' but 'between 500-700 words'. Not 'appropriate tone' but 'tone is encouraging and optimistic'. Not 'includes examples' but 'includes at least 2 real-world examples from companies mentioned in Fortune 500'.

Specify constraints explicitly. 'Don't mention X', 'Must include Y', 'Assume the reader has Z background knowledge'.

WHY THIS MATTERS (THE MECHANICS)

Remember from Session 2: LLMs work by predicting the next token based on patterns. Vague prompts activate many patterns equally. The model essentially guesses which pattern matters.

Specific prompts activate the RIGHT patterns strongly. The attention mechanism focuses. The model's predictions converge toward what you want.

It's like: Vague prompt = asking directions and being given ten different possible routes. Specific prompt = being given the one optimal route.

DELIVERY TIPS

Use three real-world use cases throughout—code, customer service, data analysis. Concrete examples stick. • Emphasize that being 'too specific' is nearly impossible. • Connect back to Session 2 mechanics: attention, pattern activation. • This is immediately actionable—students apply this in lab.

06. FEW-SHOT LEARNING: THE POWER OF EXAMPLES

~8 minutes

WHAT'S ON SCREEN

Few-shot learning concept • Zero-shot vs One-shot vs Few-shot comparison • Real example showing dramatic improvement • Pattern recognition visual

SPEAKING SCRIPT

Here's one of the most powerful techniques in prompt engineering: showing examples instead of just explaining what you want. It's called few-shot learning.

THE PRINCIPLE

LLMs learn patterns from training data. When you show the model examples of the INPUT-OUTPUT pair you want, you're activating patterns directly. The model sees: 'Oh, this is what good output looks like.' And it replicates that pattern.

It's often more effective than lengthy explanations.

ZERO-SHOT vs FEW-SHOT

Zero-shot means no examples. You just describe what you want.

Example: 'Classify the following customer reviews as positive or negative: "Great product, highly recommend."'

Few-shot means you provide examples first, then the actual task.

Example: 'Classify customer reviews as positive or negative. Here are examples:

Example 1: Review: "Great product, highly recommend." Classification: Positive

Example 2: Review: "Terrible quality, broke after one week." Classification: Negative

Example 3: Review: "It's okay, nothing special." Classification: Neutral (note: sometimes reviews are neutral)

Now classify this review: "Love it!"

Why is few-shot better? Because the model sees patterns. It learns: positive words → positive classification, negative words → negative classification, neutral words and tone → neutral classification.

When you then ask it to classify 'Love it!', it's using the learned pattern, not guessing.

REAL EXAMPLE: DRAMATIC IMPROVEMENT

Let me show you how dramatically few-shot improves quality.

Task: Extract key information from job descriptions.

ZERO-SHOT (no examples):

Prompt: 'Extract the job title, required skills, and salary range from this job description: [job description text]'

Output might be incomplete, might miss skills, might get salary format wrong.

FEW-SHOT (with examples):

Prompt: 'Extract job title, required skills, and salary range from job descriptions. Format the output as JSON. Here are examples:

Example 1:

Input: 'Senior Data Scientist wanted. Requirements: Python, SQL, 5+ years experience. Salary: \$150k-\$180k annually.'

Output: {"job_title": "Senior Data Scientist", "required_skills": ["Python", "SQL"], "experience": "5+ years", "salary":

"\$150,000-\$180,000"}

Example 2:

Input: 'Marketing Manager role. Need: digital marketing, analytics, team leadership. Pay: \$80-100k.'

Output: {"job_title": "Marketing Manager", "required_skills": ["Digital Marketing", "Analytics", "Team Leadership"], "salary": "\$80,000-\$100,000"}

Now extract from: [job description text]

Output is consistent, complete, properly formatted. The model learned the pattern by seeing examples.

HOW MANY EXAMPLES DO YOU NEED?

That's the 'few' in few-shot. Usually 2-5 examples is enough. Sometimes 1 example (one-shot) works. Rarely do you need more than 10.

General rule: Use enough examples that the pattern becomes clear. If the pattern is obvious (like classification into categories), 2-3 examples often suffice. If the pattern is complex (like specific formatting rules), maybe 4-5.

SELECTING GOOD EXAMPLES

Your examples should be representative of the kinds of inputs you'll give.

Bad example selection: If you're classifying sentiment, don't use only obvious examples ("LOVE THIS!!! = positive"). Use edge cases ("It's fine" = neutral, "Not bad" = positive despite negative phrasing).

Good example selection: Mix obvious cases with tricky cases so the model learns nuance.

WHY FEW-SHOT WORKS (THE MECHANICS)

Remember: LLMs work by predicting the next token based on patterns. When you show examples, you're showing the model exactly what pattern you want it to activate.

The model sees: 'When INPUT looks like this, OUTPUT looks like that. Here are three examples showing this pattern. Now predict the next case.'

This is incredibly efficient. The model learns from patterns, and you're showing it the pattern explicitly.

COMBINING FEW-SHOT WITH STRUCTURE

The best prompts combine structure with few-shot examples. Role, task, constraints, output format—plus examples.

Full example: 'You are a content moderator. Classify user posts as safe or unsafe (contains hate speech, violence, harassment, or illegal content). Format your response as JSON with fields "post_id", "classification", and "reason". Here are examples:

Example 1: {"post": "Love this restaurant!", "classification": "safe", "reason": "Positive sentiment, no policy violations"}

Example 2: {"post": "[racist slur] should all [violent threat]", "classification": "unsafe", "reason": "Contains hate speech and incitement to violence"}

Now classify: [user post]

This prompt has: role (moderator), task (classify), constraints (specify what unsafe means), output format (JSON), examples (showing pattern).

The output will be consistent, thoughtful, properly formatted.

DELIVERY TIPS

Few-shot learning is one of the most powerful techniques—spend real time here. • The dramatic improvement example is key—show zero-shot output vs few-shot output. • Connect to mechanics: examples are pattern-showing. • Real

moderator example shows comprehensive prompt design. • Students will use this in lab when refining their resume prompt.

07. COMMON PITFALLS

~8 minutes

WHAT'S ON SCREEN

5 Common Pitfalls listed • 'Ask the model to think step-by-step' shown as solution • Real examples of failures • Before/After comparisons

SPEAKING SCRIPT

Let me show you what kills prompts. These are the mistakes I see over and over.

PITFALL 1: AMBIGUOUS LANGUAGE

Bad: 'Write something good about AI.'

Problems: What's 'good'? Good for whom? What format? What length? What audience?

Better: 'Write a 300-word explanation of how AI is transforming healthcare, written for hospital administrators with non-technical backgrounds. Focus on benefits and ROI. Use clear language without jargon.'

Rule: Replace vague adjectives (good, nice, interesting, important) with specific criteria.

PITFALL 2: CONTRADICTORY CONSTRAINTS

Bad: 'Write a concise summary of climate change that covers all aspects thoroughly.'

Problems: 'Concise' and 'all aspects thoroughly' contradict. The model can't satisfy both.

Better: Either 'Write a concise summary (200 words) focusing on the three biggest impacts' OR 'Write a thorough analysis of all aspects of climate change (2000 words).'

Rule: Check your constraints don't fight each other.

PITFALL 3: OVERWHELMING CONTEXT

Bad: Pasting an entire 50-page document and asking 'Summarize this.'

Problems: The model has to process enormous input. Context windows limit this. Also, no guidance on WHAT to emphasize.

Better: 'Read this section [relevant excerpt, 2-3 pages]. Summarize the three key findings in bullet points. Focus on ROI implications.'

Rule: Be economical with context. Give enough information, not everything.

PITFALL 4: ASKING FOR TOO MANY THINGS AT ONCE

Bad: 'Analyze this data, identify trends, make recommendations, create a summary, suggest future research directions, and explain the limitations.'

Problems: Too many tasks. Output becomes scattered and lower quality on each task.

Better: Break into multiple prompts or prioritize. 'Analyze this data and identify the top three trends. For each trend, explain what you observed and why it matters.'

Rule: One clear task per prompt usually outperforms multiple tasks in one prompt.

PITFALL 5: FORGETTING ABOUT OUTPUT FORMAT

Bad: 'List best practices for remote work.'

Result: Paragraph format, inconsistent structure.

Better: 'List best practices for remote work. Format as a bulleted list. For each item: practice name, why it matters (one sentence), implementation tip.'

Result: Consistent, scannable, exactly the format you wanted.

Rule: Always specify output format, even if it seems obvious.

BONUS TECHNIQUE: CHAIN-OF-THOUGHT

One technique that dramatically improves complex reasoning tasks: ask the model to think step-by-step.

Bad: 'If a train leaves at 3pm going 60mph and another leaves at 4pm going 80mph, when does the second catch the first?'

The model might get it wrong by trying to calculate too quickly.

Better: 'If a train leaves at 3pm going 60mph and another leaves at 4pm going 80mph, when does the second catch the first? Think through this step-by-step: first figure out how far ahead the first train is, then calculate the speed difference, then calculate time to catch up.'

By asking the model to think step-by-step, it uses more computation, considers the problem more carefully, arrives at better answers.

WHY THESE PITFALLS MATTER (MECHANICS)

Remember from Session 2: the model's attention mechanism focuses on what seems important. Ambiguous prompts confuse the attention. The model doesn't know what to focus on. Output suffers.

Clear, specific, focused prompts make the attention mechanism focus on the right things. The right patterns activate. Output improves dramatically.

DELIVERY TIPS

These five pitfalls are practical and memorable. • Show real before/after examples for each. • Chain-of-thought is a powerful bonus technique—emphasize it. • Students have probably encountered these problems. Name the problems explicitly. • This is about debugging and improvement—frame it positively.

08. REAL-WORLD EXAMPLES

~5 minutes

WHAT'S ON SCREEN

3 Real-world use cases: Customer Support, Code Generation, Content Creation • Before/After prompts shown • Results compared • Metrics highlighted

SPEAKING SCRIPT

Let me ground all of this in real-world use. I'll show you three companies doing prompt engineering at scale and what works for them.

USE CASE 1: CUSTOMER SUPPORT (E-commerce Company)

Company: Large online retailer

Problem: 10,000 customer emails per day. Need automated responses that sound human, empathetic, and solve the problem.

Original prompt: 'Write a response to this customer email: [email]'

Result: Generic, unhelpful, low customer satisfaction (45%)

Engineered prompt: 'You are an empathetic customer service representative for [Company]. Your goal is to solve the customer's problem while making them feel valued. Analyze this customer email and respond with: 1) Acknowledgment of their issue (show you understand the specific problem), 2) Apology if appropriate, 3) Solution or next steps (specific and actionable), 4) One sentence offering additional help. Tone: warm, professional, conversational. Keep response under 150 words. [customer email]'

Result: Specific guidance on what to include. Tone specifications. Clear structure. Customer satisfaction jumped to 82%. The same model, different prompt, 37% improvement.

USE CASE 2: CODE GENERATION (Software Company)

Company: Development team using AI code generation

Problem: AI generates code that runs but is inefficient, poorly documented, or hard to maintain

Original prompt: 'Write a function to sort a list of users by their last login date.'

Result: Functional code, but missing error handling, documentation, and uses inefficient approach.

Engineered prompt: 'Write a Python function that sorts a list of user objects by their last_login_date field (datetime object) in descending order (most recent first). Include: 1) Docstring with one-line summary and example usage, 2) Type hints on parameters and return value, 3) Error handling for None values and invalid input, 4) Comment explaining the sorting approach. Assume the list could have 100,000 users, so optimize for performance.'

Result: Production-ready code. Properly documented. Error-handled. Performant. The specification matters enormously.

USE CASE 3: CONTENT CREATION (Marketing Agency)

Company: Content marketing agency

Problem: Creating variation of social media posts for A/B testing

Original prompt: 'Write Instagram captions for our product.'

Result: Generic captions that perform poorly. Low engagement. (1.2% engagement rate)

Engineered prompt: 'You are a social media strategist for [Brand]. Write 3 different Instagram captions for a post about [product]. Each caption should: 1) Hook the reader in the first 5 words, 2) Tell a brief story or ask a question related to the customer's problem, 3) Include 1-2 relevant hashtags, 4) Include a call-to-action ("Save this", "Comment below", "Link in bio"). Tone should be: friendly and relatable, not corporate. Maximum 150 characters for each. Avoid generic phrases like "Game changer" or "You won't believe." [specific context about product/audience]'

Result: Specific captions that told stories. Included hooks. Had CTAs. Engagement jumped to 4.8%. The specification,

the role definition, the style constraints—all mattered.

PATTERN ACROSS ALL THREE

Notice what's consistent: Structure (role, task, constraints, format). Specificity (not 'good', but specific quality criteria). Examples (sometimes). Clear success metrics (customer satisfaction, code quality, engagement rate). The engineering isn't about being clever. It's about being systematic.

DELIVERY TIPS

Use real companies/scenarios—even if anonymized. • Show specific numbers (37% improvement, 4.8% engagement) to make it visceral. • Point out the commonalities: all three follow structure + specificity principles. • Emphasize that these are real outcomes—not theoretical. • Students will apply these patterns in their lab.

09. PROMPT TEMPLATES & PATTERNS

~5 minutes

WHAT'S ON SCREEN

5 Reusable Templates: Summarization, Classification, Generation, Analysis, Extraction • Each template shown with variables • Real examples • Downloadable reference format

SPEAKING SCRIPT

Here are five templates you can use and adapt for common tasks. These are battle-tested patterns that work reliably.

TEMPLATE 1: SUMMARIZATION

Use when: Condensing long content into key points

Template: 'You are a [role: expert, analyst, writer]. Summarize the following [source type: article, report, email, document] into [number] key points. Each point should be [specific criteria: 1-2 sentences, include specific metrics, focus on impact]. Avoid: [what to exclude]. Format: [bullet points / numbered list / paragraph]. [Source text]'

Example: 'You are a business analyst. Summarize this quarterly earnings report into 5 key findings. Each finding should include the specific metric and whether it indicates growth or decline. Avoid technical jargon. Format as a numbered list. [report text]'

TEMPLATE 2: CLASSIFICATION

Use when: Categorizing items into predefined categories

Template: 'You are a [role]. Classify each [item type] into one of these categories: [list categories]. Use these criteria: [specific decision rules]. Format your response as [JSON / list / table] with fields [specify fields]. Examples: [1-2 examples]. [Items to classify]'

Example: 'You are a content moderator. Classify each user post into: safe, needs review, or unsafe. Posts containing [specific prohibited content] should be classified as unsafe. Format as JSON with fields "post_id", "classification", "confidence" (high/medium/low). Here are examples: [examples]. [Posts to classify]'

TEMPLATE 3: GENERATION

Use when: Creating new content

Template: 'You are a [role]. Generate [number] [content type] with the following specifications: Audience: [describe audience], Tone: [specific tone], Length: [specific length], Key points to include: [list], Format: [format], Constraints: [what to avoid]. [Additional context]'

Example: 'You are a marketing copywriter. Generate 3 different email subject lines for a product launch email. Audience: non-technical small business owners, Tone: friendly and curious, Length: 50-70 characters, Key point: product saves time, Constraints: avoid ALL CAPS, avoid clickbait phrasing, avoid mentioning price. Our product helps teams automate workflows.'

TEMPLATE 4: ANALYSIS

Use when: Examining data or information systematically

Template: 'You are a [role]. Analyze the following [data type] to identify: [specific things to look for]. For each finding, provide: [what to include in response]. Think step-by-step. Format your response as: [specific format]. [Data to analyze]'

Example: 'You are a business data analyst. Analyze this sales data to identify: trends, anomalies, and opportunities. For each finding, provide: what you observed, why it matters, recommended action. Think step-by-step through the data. Format as a bulleted list. [sales data]'

TEMPLATE 5: EXTRACTION

Use when: Pulling specific information from unstructured text

Template: 'You are a [role]. Extract the following information from [source]: [list what to extract]. Format output as: [specify format]. If information is not present, indicate "Not specified". Here are examples: [examples]. [Source text]'

Example: 'You are a contract analyst. Extract the following from this employment contract: job title, salary range, start date, reporting manager role, benefits listed. Format as JSON. If a field is not mentioned, put "Not specified". Example: {"job_title": "Senior Engineer", "salary_range": "\$150k-\$180k", ...}. [Contract text]'

HOW TO USE THESE TEMPLATES

Copy a template. Replace the bracketed sections with your specific details. The variable parts are marked with brackets. The fixed structure is proven to work.

You can combine templates. You might have a 'summarization' task followed by a 'classification' task. Use both templates in sequence.

These templates embed all the principles we've discussed: role, task, constraints, output format, examples where helpful.

FINAL NOTE ON TEMPLATES

Templates are starting points, not dogma. Adapt them. If a task needs a different structure, use it. But these five templates cover 80% of common use cases. Master them and you can build prompts for almost anything.

DELIVERY TIPS

Templates are immediately practical—students will use them. • Keep it simple: copy-paste, customize, done. • Show 2-3 full examples so students see what a customized template looks like. • Emphasize these are patterns, not formulas. • Students can bookmark these for reference.

10. LAB EXERCISE OVERVIEW

~3 minutes

WHAT'S ON SCREEN

Lab title: 'Craft Your Production-Ready Prompt' • 5-step process with visual flowchart • Deliverables checklist • Success criteria highlighted

SPEAKING SCRIPT

Alright, let's put everything into practice. Your lab is more advanced than Session 2. You're not just testing variations. You're systematically engineering a prompt to be production-ready.

STEP 1: BASELINE (5 minutes)

Start with your Session 1 resume and the prompt you used to generate it. Evaluate: what's working? What's not? Be specific. Document your baseline observations.

STEP 2: APPLY STRUCTURE (7 minutes)

Rewrite your prompt using the four-part structure: Role, Task, Constraints, Output Format. You should have: clear role (maybe 'professional resume writer'), explicit task (generate resume with these specs), specific constraints (length, tone, sections), and exact output format (professionally formatted plain text, or Markdown, or whatever).

STEP 3: ENHANCE WITH CLARITY & SPECIFICITY (7 minutes)

Go through your engineered prompt. Find any vague language. Replace it with specific criteria. Instead of 'good resume', specify: 'uses action verbs, includes quantifiable metrics, keeps each bullet to one line, uses parallel structure'.

STEP 4: ADD EXAMPLES (7 minutes)

Create 2-3 examples of what good output looks like. This could be: a resume you like formatted exactly how you want, or sections from other resumes that exemplify the style you want. Include these examples in your prompt so the model learns the pattern.

STEP 5: TEST & ITERATE (6 minutes)

Run your engineered prompt. Evaluate the output. Does it match your specs? If yes, document what worked. If no, identify what's missing. Refine one aspect. Run again. Document the improvement.

DELIVERABLES

You'll document: your baseline prompt and evaluation, your engineered prompt (with role, task, constraints, format clearly labeled), the examples you added, your test output, your iteration notes (what you changed and why), and a final reflection: what was the biggest improvement? What would you do next?

This is about becoming systematic in your prompt engineering. Not trial-and-error. Methodology.

DELIVERY TIPS

This is more rigorous than Session 2—emphasize that. • Five clear steps make it manageable. • Deliverables checklist ensures nothing is missed. • Systematic iteration over random tweaking. • This is professional-level prompt work.

11. KEY TAKEAWAYS FROM SESSION 3

~5 minutes

WHAT'S ON SCREEN

6 Core Principles listed • Each with supporting insight • Real-world impact noted • Bridge to Session 4

SPEAKING SCRIPT

Takeaway One: Prompt Engineering is Systematic, Not Magical

There are patterns. Structures that work. Principles that apply. Master the structure and you can engineer any prompt.

Takeaway Two: Structure Matters

Role, Task, Constraints, Output Format. Use this framework. It organizes your thinking and improves results dramatically.

Takeaway Three: Specificity is Your Friend

You can't be too specific. Replace vague language with measurable criteria. The more specific, the better the output.

Takeaway Four: Examples Are Powerful

Showing the model what you want often beats explaining. Few-shot learning activates the right patterns directly. Always consider using examples.

Takeaway Five: Common Pitfalls Are Avoidable

Ambiguous language, contradictory constraints, overwhelming context, too many tasks, forgotten format specifications. Know these. Avoid them. Your prompts will be dramatically better.

Takeaway Six: Templates Accelerate Your Work

Five templates cover 80% of use cases. Summarization, classification, generation, analysis, extraction. Learn them. Use them. Adapt them.

These six principles—structure, specificity, examples, avoiding pitfalls, templates, systematic approach—are everything you need to engineer prompts like a professional.

DELIVERY TIPS

These six takeaways are immediately practical. • Students should remember them long-term. • They anchor everything taught in Session 3. • Connect each to real-world impact: better customer service, production-ready code, higher engagement.

12. COMING UP NEXT

~2 minutes

WHAT'S ON SCREEN

Dark blue background • Title: 'Session 4: Structured Outputs & Parsing' • Preview bullets • Closing remarks

SPEAKING SCRIPT

Session 4 builds on what we've done. We've learned how LLMs work (Session 2), and we've learned how to write prompts that work well (Session 3).

Now: what if you need the output in a specific format you can feed into another system? What if you need JSON, or CSV, or XML? What if the output needs to be reliably parseable?

That's structured outputs. We'll learn how to engineer prompts so the model generates output in formats that downstream tools can consume automatically. This unlocks: building AI pipelines where one AI's output becomes another AI's input. Building systems where human and AI work in tandem. Building reliable, scalable AI workflows.

In the lab, you'll generate JSON, parse it, extract specific fields, and verify it's valid. You'll see firsthand why structured outputs matter for real systems.

Thank you all for your engagement today. Prompt engineering is a skill you'll use every time you work with LLMs. Master it now and it compounds. See you in Session 4.

DELIVERY TIPS

Keep this brief and energizing. • Session 4 connects naturally: now that you can write good prompts, make them output structured data. • End on an upbeat note. Students should feel like they've learned something powerful and useful.